Booking Modernization Assessment

Candidate: Jack Huynh Target role: PhoenixDX .NET Backend Engineer Deliverable: 6-page technical design PDF Scope: Extract booking capability from a legacy Expedia-like travel platform into a .NET booking microservice.

Page 1 - Executive Summary

Summary

This proposal extracts booking lifecycle ownership from a legacy system into a .NET 8 booking microservice. The service owns booking commands, status transitions, idempotency, event publication, reconciliation, and production observability. It does not try to absorb payment, inventory, traveller profile, supplier catalog, communications, or reporting ownership.

The Expedia-like domain matters because a booking can contain stays, flights, cars, packages, and activities. The service should coordinate travel booking state and keep stable references/snapshots, while specialist systems keep their own source-of-truth responsibilities.

Assumptions And Constraints

Area	Position
Platform	.NET 8 / ASP.NET Core style service, JSON APIs, structured telemetry.
Legacy	Existing system remains during migration; no big-bang rewrite.
Ownership	Booking service becomes canonical for migrated booking flows only.
Consistency	Eventual consistency; no distributed transaction across booking, payment, inventory, supplier, broker, or legacy DB.
Delivery	Design-only PDF. No backend implementation, Docker, CI, or runtime harness.

Outcomes

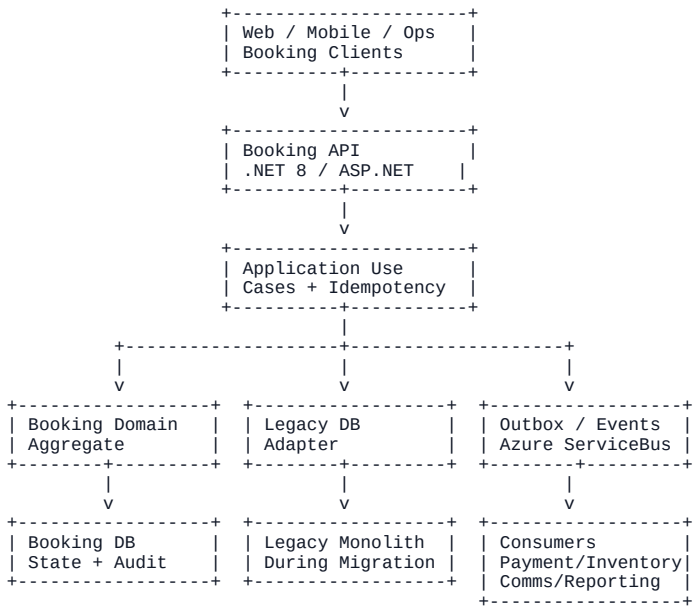
Outcome	Design response
Prevent duplicate bookings	Mandatory idempotency keys and stored response snapshots.
Avoid lost events	Transactional outbox written with booking state.
Handle partial failure	Explicit states, compensation, and reconciliation.
Keep migration safe	Legacy adapter, ownership markers, feature flags, flow-by-flow cutover.
Operate in production	Logs, metrics, traces, alerts, dashboards, and investigation paths.

Page 2 - Booking Microservice Structure

Service Boundary

Booking service owns	Outside boundary
Create, confirm, cancel, expire, fail booking workflows.	Payment authorization/settlement.
Booking aggregate, status, version, line references, audit snapshots.	Inventory/availability source of truth.
Booking rules, duplicate prevention, state transitions.	Traveller profile, supplier catalog, communications.
REST API, idempotency store, outbox, reconciliation.	Legacy monolith internals and historical reporting jobs.
Legacy sync adapter during transition.	Legacy schema as a domain model.

High-Level Booking Architecture



Layer	Responsibility
API	Auth, validation, Problem Details errors, correlation ID, request/response mapping.
Application	Use cases, transaction boundaries, idempotency checks, repository/outbox writes.
Domain	Booking aggregate, line model, status transitions, business invariants.
Infrastructure	EF Core persistence, legacy gateway, broker client, workers, telemetry.

Migration Approach

Stage	Legacy DB role	Booking service role
Observe	Source of truth	Shadow reads/mapping comparison.
Strangler facade	Still canonical	Owns new API contract, idempotency, outbox audit.
Flow cutover	Read compatibility and selected sync	Canonical for migrated create/confirm/cancel flows.
Service-owned	Archive/reporting input	Canonical state and event history.

Rollback routes not-yet-finalized commands back to the monolith. Already accepted service-owned bookings stay service-owned and reconcile forward.

Page 3 - API Contract Summary

API Principles

Principle	Contract choice
Use-case APIs	Commands map to booking workflows, not table updates.
Retry safety	Mutating endpoints require Idempotency-Key.
Traceability	All requests accept or generate X-Correlation-Id.
Eventual consistency	Commands may return 202 Accepted; clients read current status.
Legacy isolation	Public contract uses service-owned IDs and statuses.

Core Endpoints

Endpoint	Purpose	Success
POST /api/bookings	Create booking from traveller and booking lines.	201 or 202
GET /api/bookings/{bookingId}	Read canonical booking state.	200
GET /api/bookings?...	Search by traveller, status, date window.	200
POST /api/bookings/{bookingId}/confirm	Confirm after required outcomes.	200 or 202
POST /api/bookings/{bookingId}/cancel	Cancel by traveller, operator, or system.	200 or 202
POST /api/bookings/{bookingId}/expire	Expire stale pending booking or quote/hold.	200 or 202

Booking Line Model

Field	Notes
type	stay, flight, car, package, or activity.
supplierRef, quoteRef	External references, not service ownership.
priceSnapshot	Captures currency, total, tax/fee, and quote time for audit.
startDate, endDate	Required for dated travel products.
details	Type-specific details such as room, flight segments, vehicle class, package components, or activity session.

Error And Validation Posture

Case	Response
Missing idempotency key	400 Bad Request
Unknown booking	404 Not Found
Version/state conflict	409 Conflict
Valid JSON but invalid business request	422 Unprocessable Entity
Downstream unavailable and command cannot be accepted safely	503 Service Unavailable

Errors use Problem Details with correlationId and field-level codes. Legacy IDs may appear only as optional migration/support references.

Page 4 - Events And Eventual Consistency

Event Envelope

Field	Purpose
eventId	Stable ID and broker message ID for dedupe.
eventType	Lifecycle fact such as BookingConfirmed.
schemaVersion	Starts at 1.0; breaking changes use a new major version or event type.
occurredAt	State-change time, not publish time.
correlationId, causationId	Trace request and command/event that caused this fact.
aggregateId, aggregateVersion	Booking ID and monotonic version for ordering/stale detection.

Published Events

Event	Emitted when	Likely consumers
BookingRequested	Create command accepted and persisted.	Payment, inventory, trip projection, reporting, legacy sync.
BookingConfirmed	Booking reaches confirmed state.	Communications, support, reporting, legacy sync.
BookingCancelled	Booking enters cancelled state.	Refund workflow, inventory release, communications, projections.
BookingFailed	Terminal failure from payment, inventory, supplier, validation, or sync outcome.	Compensation, support, reporting, communications.
BookingExpired	Pending booking or hold/quote expires.	Inventory cleanup, payment cleanup, projections.

Consistency Flow

```
Command
  -> Validate idempotency
  -> Apply booking transition
  -> Commit booking state + idempotency result + outbox event
  -> Outbox worker publishes event
  -> Consumers process with inbox/dedupe
  -> Reconciliation repairs drift
```

Tradeoff

The design accepts temporary lag between booking, payment, inventory, legacy DB, communications, and reporting. In return it avoids distributed transactions, makes pending states explicit, and gives operations clear replay and repair paths.

Page 5 - Idempotency, Edge Cases, Reliability

Idempotency Strategy

Command	Scope
Create	Caller + CreateBooking + key.
Confirm	Caller/worker + ConfirmBooking + booking ID + key.
Cancel	Caller/worker + CancelBooking + booking ID + key.
Expire	Worker + ExpireBooking + booking ID + key.

The idempotency record stores key scope, canonical request hash, command type, booking ID, response snapshot, status, expiry, caller, and correlation ID. Same key and same payload returns the stored response. Same key and different payload returns 409 Conflict.

Reliability Patterns

Pattern	Reason
Optimistic concurrency	Prevent confirm/cancel/expire races from committing conflicting transitions.
Transactional outbox	Prevent committed booking state without a publishable event.
Consumer inbox/dedupe	Tolerate duplicate broker delivery and event replay.
Reconciliation worker	Compare booking, payment, inventory, legacy, and outbox/broker state.
Dead-letter handling	Isolate poison events and preserve metadata for replay.

Required Edge Cases

Edge case	Handling
Duplicate booking submission	Same idempotency key returns original bookingId; no second booking or logical event.
Payment succeeds but booking fails	Booking stays pending or moves to Failed; payment refund/void compensation starts.
Inventory unavailable after request	Booking moves to Failed; traveller can retry with fresh quote/key.
Cancellation while payment pending	Cancellation wins if policy allows; late payment becomes refund/void input.
Legacy sync timeout	Service-owned booking is not rolled back; mark sync pending/failed and reconcile.

Terminal states are not silently overwritten by late events. Corrections require explicit administrative workflow outside this assessment scope.

Page 6 - Observability And AI Usage

Observability Plan

Area	Signals
Logs	Structured JSON with bookingId, travellerId, correlationId, causationId, idempotency key/hash, eventId, eventType, aggregate version, optional legacy ID.
Metrics	Creation/confirmation/cancellation rates, failed bookings, outbox backlog/latency, legacy sync failures, dependency timeouts, reconciliation mismatches.
Traces	API request, use case, DB transaction, legacy adapter, payment/inventory calls, outbox publish, consumer handling.
Alerts	High failure ratio, growing outbox backlog, dead-letter growth, legacy sync spike, reconciliation mismatch spike, dependency degradation.
Dashboards	Booking lifecycle health, event publishing health, legacy migration health, dependency health.

Operational Targets

Target	Objective
API availability	99.9% monthly for booking command/read endpoints.
Command latency	95% acknowledgements under 500 ms when async work continues.
Event publish delay	95% outbox events published within 30 seconds; 99% within 2 minutes.
Reconciliation freshness	Critical mismatches detected within 15 minutes; hourly full cutover checks.

AI Usage Declaration

AI Tools Used:
OpenAI Codex / ChatGPT-style assistant was used during this exercise. I also used project-scoped .NET agent skills installed under .codex/skills, including .NET architecture, modern C#, testing, coverage, and project setup related skills, to guide review checklists and terminology.

Sections AI-Assisted:
Roadmap, architecture structure, API contract, event model, idempotency and reliability design, observability plan, AI usage wording, final PDF compression, and completeness checks.

Manually Validated:
I manually reviewed the assessment constraints, Expedia-like travel domain framing, .NET 8 microservice boundary, API and event consistency, idempotency/outbox strategy, edge cases, observability practicality, PDF page limit, and final submission quality.

Preventing Blind AI Usage:
In a backend team, I would require human design review, source-backed assumptions, architecture decision records for major decisions, tests for generated implementation code, security/privacy review, production-readiness checks, and a rule that no unreviewed AI output is merged into production work.

Closing Position

This design favors a pragmatic strangler migration: clear booking ownership, retry-safe commands, reliable events, explicit eventual consistency, and production support from day one.